AGENTWARD + MUSEVERIFY

Evidence-driven security for Python, JavaScript and TypeScript

Project guide / current-stage edition

A Meta-primary harness that inspects source, reproduces supported vulnerabilities in Docker, proposes repairs and requires independent verification before accepting them.

99 tests Passed; no skips	**13 live Node targets** 8 verified repairs	**24 self-play scenarios** Neutralized; 0 survivors

Current release
Seven-phase verification for Python, JavaScript and TypeScript. Meta Spark and Wonder are mandatory; optional providers extend the review quorum.

Evidence snapshot
Live Node corpus: 8 REMEDIATED, 4 FALSE_POSITIVE, 1 INCONCLUSIVE. Historical checks: 78 classifier cases and 25 remote acceptance groups. Scope and dates appear on pages 17-18.

Application snapshot **2d52f29**
1 November 2026

01 / PROJECT GUIDE

Problem and core value

Candidate alerts need evidence before repair. Coding agents need containment when they inspect untrusted source or propose changes.

Two problems, one controlled workflow

Automated findings can consume maintainer time without proving a reachable bug. A model-generated patch can also change behavior or introduce a new weakness. Meanwhile, comments and literals can carry indirect instructions aimed at credential access, destructive actions or fabricated audit results.

Area	Current stage
AgentWard	Source-aware ingress, proposed-tool-action policy, additive operator restrictions and checkpoint rewind. It protects the controlled workflow.
MuseVerify	Isolated exploit reproduction, Meta-led repair, independent review, CodeShield, AST tripwires, self-play and functional regression.
Core rule	Model advice can propose a repair. Only the orchestrator can accept it after all required evidence gates pass. Model judgments cannot waive deterministic controls.
Evidence	Protected Docker traces and artifact hashes identify tested behavior. Static findings, fixture modes and live verified repairs stay distinct.

Read this guide by task

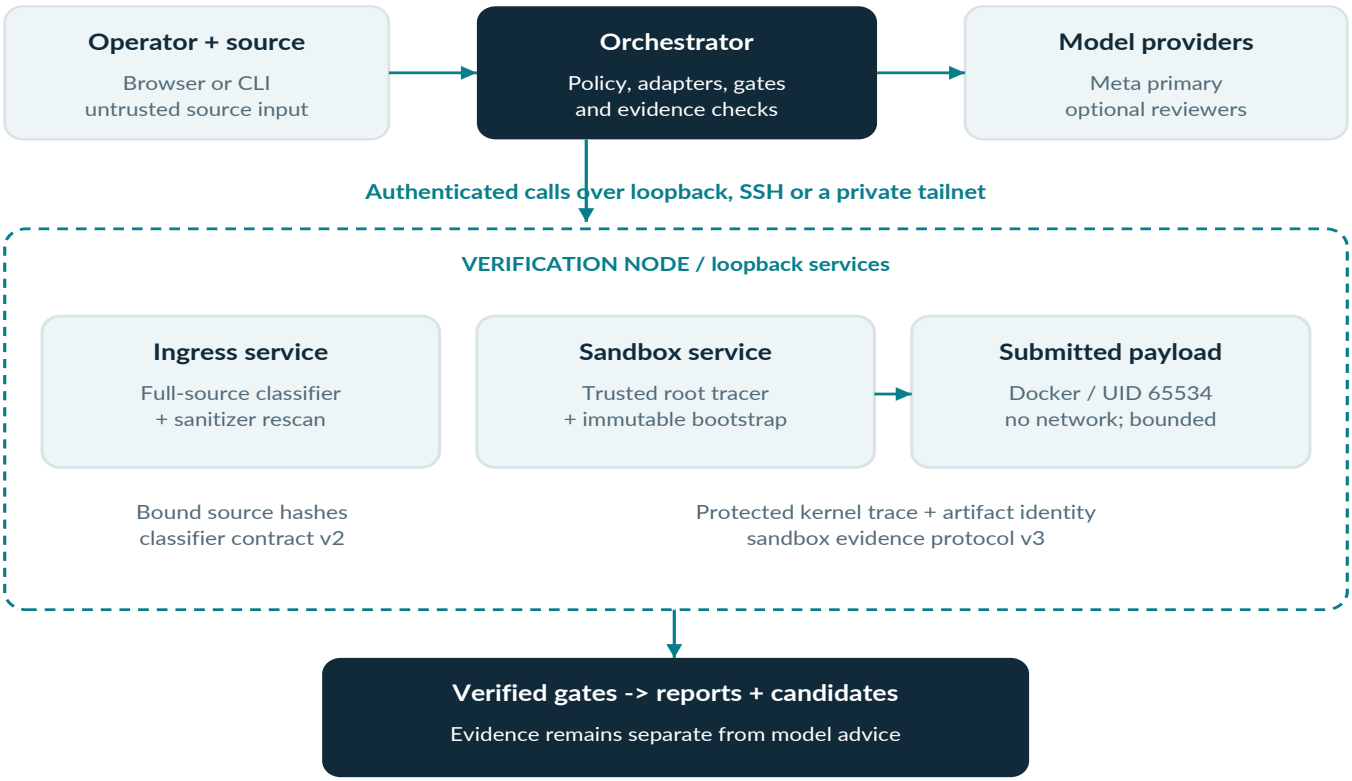
Task	Pages
Understand the architecture and supported inputs	3-5
Understand detection, policy and repair acceptance	6-9
Run and operate the system	10-15
Review implementation, results and coverage limits	16-20

README.md; docs/SKILLS.md; docs/INGRESS_GUARD.md; docs/DEPLOYMENT_V3.md

02 / PROJECT GUIDE

Architecture and trust boundaries

The workstation coordinates policy and verification. Submitted source and model output remain untrusted throughout the workflow.



What must be trusted

The operator configuration, application services, selected provider identities, Docker host and kernel, runtime image, bootstrap and tracer form the trusted base. A compromised host or service can undermine the evidence it produces.

What evidence establishes

Authentication controls who can request work. Hashes bind results to particular source and harness artifacts. Protected kernel traces record selected execution behavior. The orchestrator combines that evidence with positive controls, reviews and regression results.

Two different contracts
Classifier contract v2 binds model, policy, language and source hashes. **Sandbox protocol v3** binds protected runtime evidence, artifact hashes, language, runtime and filename. Both must match the workstation.

docs/THREAT_MODEL.md; README.md; service_security.py; sandbox_protocol.py

03 / PROJECT GUIDE

The seven-phase workflow

Every supported language uses the same orchestrator. A failed or unavailable required gate prevents a verified repair.

Phase	Work performed	Acceptance evidence
1 / Ingress Shield	Classify complete source and sanitize injected instructions.	Full coverage, rescan after changes, exact model/policy contract and source hashes.
2 / Syscall Sandbox	Reproduce supported exploits with deterministic probes in Docker.	Protected strace evidence, artifact identity and completed behavioral probes.
3 / Runtime Hooks	Apply screening, ensemble policy checks and trajectory monitoring.	Valid unanimous votes and the current operator policy revision; violations trigger rewind.
4 / Patch Verification	Synthesize a repair; inspect contracts, static paths, reviews, CodeShield and tripwires.	All applicable structural, scanner and independent-review gates pass.
4.5 / Adversarial Self-Play	Replay bounded mutated and multi-step scenarios.	Original-code positive controls succeed; no surviving exploit in the candidate; valid evidence.
5 / Regression	Replay exploits, normal inputs and decoy probes.	Normal result preserved; exploit blocked; decoy triggers the logger, normal probe does not.
5.1 / Semantic Hunter	Inspect related modules and semantic paths with Wonder-led analysis.	Static findings and unresolved assumptions remain visible, separate from runtime proof.

Concurrency and reporting

Phase 2 overlaps with graph construction and the initial policy baseline. The pipeline runs at most three containers concurrently. Passing that baseline cannot replace the later sandbox, patch or regression evidence.

A valid Phase 2 BENIGN result still requires Phase 3 policy review before FALSE_POSITIVE. For Node, static findings outside SQL/command probe coverage remain INCONCLUSIVE. Benign controls skip patch synthesis and later repair stages.

README.md; main_pipeline.py; docs/LANGUAGES.md

04 / PROJECT GUIDE

Supported languages and targets

Language support means a defined parser, execution harness and verification contract. It does not imply arbitrary project builds.

Language	Files	Supported verification scope
Python	.py	Existing Python parsing, graph analysis and benchmark probes.
JavaScript	.js / .mjs / .cjs	Single-file Node ES/CommonJS modules with compatible exported SQL/command functions.
TypeScript	.ts / .mts / .cts	The same Node probes with erasable syntax. Original source executes in Node 24; scanning uses AST-based type erasure.

Node target contract

Node targets support named top-level functions and arrow functions in ES/CommonJS exports. All exports (maximum eight) must accept a single scalar SQL/command probe argument and return serializable results. Ambiguous or partial probe coverage is rejected.

Execution uses built-in dependencies such as node:sqlite and node:child_process. Every export must return successfully for **agentward_normal**; other business contracts need additional probes. Node always runs in Docker.

TypeScript behavior

Tree-sitter parses without execution. Type-only nodes are erased and reparsed as JavaScript for CodeShield. Original TypeScript executes in Docker. Supported syntax includes annotations, interfaces, type aliases, generics and as-assertions; there is no type checking.

Result	Meaning
UNSUPPORTED_LANGUAGE	Extension outside the registry: for example Go, Java, Rust, JSX or TSX.
INVALID_SOURCE	Syntax is invalid for the selected language.
UNSUPPORTED_TARGET	The supported language has no fully compatible Node probe surface.

Outside the current target contract
Browser apps, npm dependencies, multi-file execution, constructed classes and project compiler plugins are unsupported. TypeScript enums, namespaces, decorators and unsupported transformations cannot pass the complete chain.

docs/LANGUAGES.md; language_adapters.py; ecmascript_harness.py

05 / PROJECT GUIDE

Ingress: detection with context

The ingress guard detects instructions aimed at manipulating the audit. It is a separate task from finding vulnerabilities in the program.

How the current guard works

The **Horizon-Labs/prompt-injection-guard-small** model, pinned at revision **5ed691c** under Apache-2.0, receives bounded parser facts followed by the complete source. The prefix describes language, declared names and syntax-node kinds; it never declares the source safe. Valid syntax and filenames cannot bypass classification.

Control	Current behavior
Coverage	512-token windows with 128-token overlap; the highest injection score controls the decision. Risk threshold: 0.40.
Work bound	At most 256 windows per original or sanitized input. Over-limit or incomplete coverage is denied, without silently truncating source.
Sanitization	Recognizes explicit role-directed bypass or fabricated-audit instructions in comments/literals, including decoded base64. Modified source receives a full rescan.
Model integrity	Five artifacts are pinned and SHA-256 verified. Safetensors, no remote code, no inference-time downloads; provisioning is explicit.
Evidence	Contract v2 binds the model revision and policy identity, language and original/sanitized hashes. Stale or mismatched responses cannot authorize a live run.

What “detection remains probabilistic” means

The model assigns a learned risk score. It can occasionally flag harmless code or miss an unfamiliar instruction attack. **78 of 78 regression cases passed** means those specific cases behaved as expected; it does not mean every future input will be classified correctly.

The 0.40 threshold is a decision setting, not a measured 40% chance that a program is unsafe. Ingress ALLOW means the file can proceed to the next checks; a vulnerable SQL or command function may correctly pass ingress and then fail sandbox verification.

Short HTTP 429 capacity delays allow at most three identical client requests; exhausted retries remain unverified. All 13 current Node corpus files received unchanged-source ALLOW, with a maximum measured injection score of 0.017127.

docs/INGRESS_GUARD.md; ingress_guard.py; security/ingress-model.lock.json

06 / PROJECT GUIDE

Protected runtime evidence

The remote sandbox executes submitted code under a fixed, constrained runtime and keeps the evidence outside the payload's control.

Boundary	Enforced control
Identity	Bearer authentication and bounded request schemas. Protocol v3 binds source/harness SHA-256, language, runtime and target filename.
Privileges	Trusted root tracer and immutable bootstrap start first. Payload UID/GID: 65534; supplementary groups cleared; effective/permitted capabilities empty.
Filesystem / network	No container network. Read-only root, source inputs and bootstrap; bounded 64 MiB workspace and 16 MiB temporary storage.
Trace protection	Root-owned kernel trace in a POSIX-ACL-protected directory. Payloads cannot read, overwrite, unlink or replace host-backed evidence.
Resources	256 MiB memory/swap ceilings, CPU/PID limits and bounds on time, output, trace bytes and event counts. Overflow and timeout deny evidence and trigger cleanup.
Runtime choice	Fixed python3/node allowlist. Python isolated startup; Node loader environment variables cleared. No arbitrary caller-selected interpreter.

Conditions for accepted execution evidence

The client requires matching protocol and artifact identity plus complete protected traces and valid behavioral results. Missing traces, tampering, truncation, abnormal completion and unsupported ownership/ACL setup deny evidence. Docker failure cannot trigger production host execution.

Deployment requirements
The supported node uses rootful Linux Docker without user-namespace remapping and an ACL-capable temporary filesystem. Rootless or remapped configurations are not validated.

Evidence is authenticated and hash-bound, not digitally signed. Selected syscalls and payload-produced probe lines cannot prove all possible behavior; the other acceptance gates remain necessary.

docs/THREAT_MODEL.md; sandbox_server.py; sandbox_protocol.py

07 / PROJECT GUIDE

Runtime policy and workspace rewind

Phase 3 reviews the controlled tool workflow using the completed ingress and original sandbox evidence.

Screen actions before authorizing continuation

agentward_stage3.py checks proposed action text for destructive commands, secret-exfiltration patterns and high-entropy strings, and tracks an estimated token budget. Known credential values are redacted from action text. Meta Spark leads concurrent policy and trajectory reviews with the selected independent auditors.

Mechanism	Scope
Tool hook	enforce_permissions.py evaluates command-bearing JSON arguments for a PreToolUse integration. These are application hooks, not kernel syscall interception.
Policy baseline	A checklist may be prepared during Phase 2. Final Phase 3 authorization requires the observed Docker evidence; preparation is not permission.
Checkpoint / rewind	The guard copies top-level workspace files and writes SHA-256 manifest entries. trigger_rewind restores copies and removes new top-level files.
Trajectory monitor	Drift, forbidden actions, missing review or stale policy can deny continuation. Detected violations can halt the session and trigger rewind.

Add operator restrictions during a run

```
POST /api/steer
{"constraint":"Forbid subprocess calls"}
```

Restrictions are additive and shared across dashboard batch workers. Each revision invalidates older in-flight approval. Recognized rules also have AST checks; broader natural-language rules depend on model review.

> **What rewind does not promise**
> The checkpoint covers the disposable workspace, not arbitrary host effects, nested files or Git history. strace observes submitted code inside Docker. The tool hook must be integrated to mediate another agent's actions.

agentward_stage3.py; enforce_permissions.py; session_policy.py; main_pipeline.py

08 / PROJECT GUIDE

How a repair earns acceptance

A proposed patch stays a candidate until independent checks agree with the observed behavior.

Preserve the public contract

Patch checks preserve relevant function interfaces and sync/async behavior. Node checks also compare exported parameters and type signatures. Model-generated text cannot waive structural rules, scanning or current operator restrictions.

Use independent evidence at each step

Gate	What it contributes
Policy and review	Valid votes from the selected reviewers, bound to current additive operator constraints. Missing or malformed reviewer output denies approval.
Static checks and CodeShield	Contract inspection, call/taint findings and scanner results. TypeScript is erased and reparsed for JavaScript rules; execution uses the original source.
Structural tripwire	A reachable AST decoy equality guard calls a logger such as _tripwire(). A comment mentioning a canary cannot satisfy this check.
Adversarial replay	Deterministic mutations and bounded scenarios run against original and candidate code. Original-code positive controls must reproduce the relevant behavior.
Regression	The original exploit is blocked, tested normal results are preserved, and the tripwire fires only for decoy probes.

Bounded self-play

Node tests three deterministic scenarios: syntax mutation, stored-value replay and URL-encoding replay. All three must be neutralized with valid evidence and successful original-code controls. Python also supports bounded model-proposed scenarios executed by deterministic harnesses.

Zero surviving mutations alone is insufficient. Unsupported or unproven scenarios are not counted as neutralized. Storage adapters test replay robustness, not arbitrary native persistence.

Review the exact diff before integration
Accepted files and diffs go to **patches/<run-id>/**. Run **git apply --check** against the unchanged source before applying a diff; this is an integration step, not an automatic pipeline gate. Corpus acceptance applied all eight diffs in temporary directories and matched the saved patch hashes.

README.md; museverify_stage4.py; main_pipeline.py; docs/LANGUAGES.md

09 / PROJECT GUIDE

Providers and operator control

Meta remains mandatory for every live run. Optional providers add independent review when they pass pre-flight.

muse-spark-1.3 leads runtime policy and patch synthesis. **muse-wonder-1.0** leads deeper semantic analysis. In Meta-only operation, the two native models cross-check each other.

Mode	Selected participants
auto	Meta primary plus every healthy secondary.
all_three	Meta, Anti-Gravity and Custom; notices identify unavailable secondary providers.
meta_plus_agy	Meta plus verified Anti-Gravity.
meta_plus_custom	Meta plus verified Custom BYOK.
meta_only	Spark and Wonder cross-check each other.

When no requested secondary passes pre-flight, Meta's other native model supplies the independent review. If Meta is offline, a live run cannot proceed. A selected reviewer that later fails cannot silently become an approval.

Pre-flight and configuration

Provider discovery runs concurrently and caches dashboard status for 60 seconds. A forced refresh reloads .env. Each run retains its successful discovery snapshot while configuration is unchanged; task failures still deny approval. Configure META_API_KEY, ENSEMBLE_MODE and optional Anti-Gravity or Custom endpoint/model credentials locally.

Optional high-end co-auditors

Anti-Gravity tries a supported selected model, then a deduplicated cascade: gemini-4-argon, claude-opus-5-5, claude-sonnet-5.5 and gemini-3.8. Quota failures are recorded during pre-flight. Custom BYOK supports compatible Responses and Chat Completions endpoints, including a /v1 base.

> **Requested topology versus verified participation**
> all_three requests all three providers but excludes an unhealthy secondary with an explicit notice. It does not waive the required independent review. A selected reviewer failure cannot silently become approval.

README.md; ai_provider.py; agentward_stage3.py; dashboard.py

10 / PROJECT GUIDE

Configure providers and node access

Use .env.example as the schema. Provider credentials stay on the workstation; the verification node uses its separate shared bearer token.

Configuration	Purpose
META_API_KEY META_API_BASE_URL	Mandatory Meta key; default API base https://api.meta.ai/v1.
ENSEMBLE_MODE	auto, all_three, meta_plus_agy, meta_plus_custom or meta_only.
AGY_BIN AGY_SELECTED_MODEL	Optional CLI path and preferred supported model; default gemini-4-argon.
CUSTOM_API_URL CUSTOM_API_KEY CUSTOM_MODEL_NAME	Compatible HTTP endpoint, BYOK credential and a model identifier supported by that endpoint.
CUSTOM_REASONING_EFFORT CUSTOM_SERVICE_TIER	Optional model/endpoint features. Effort defaults to medium; service tier is empty unless configured.
AI_TIMEOUT_S AGY_TIMEOUT_S	Task deadlines default to 120 / 180 seconds. Custom pre-flight has a five-second deadline.
AGENTWARD_NODE_TOKEN	Same random token as the node configuration, at least 32 characters.
AGENTWARD_INGRESS_URL AGENTWARD_SANDBOX_URL	Authenticated /scan-ingress and /execute endpoints over loopback, SSH or private tailnet access.

Provider readiness is measured

GET /api/providers reports availability and topology without credential values. Meta pre-flight tests both native models. Custom requires HTTP 200 with nonempty, complete content; placeholder keys, auth errors, timeouts and missing endpoints exclude it.

> **Configuration changes require fresh evidence**
> Use GET /api/providers?force_refresh=true after editing provider configuration. Restart the dashboard after node endpoint changes. Unknown model pricing remains unknown; token telemetry is not an invoice or a promised cost reduction.

ai_provider.py; .env.example; service_security.py

11 / PROJECT GUIDE

Start a supported audit

Use the local dashboard or CLI after configuring the providers and authenticated verification-node endpoints.

1 / Prepare the workstation

```
python3 -m venv venv
. venv/bin/activate
python -m pip install -r requirements-dev.txt
cp -n .env.example .env
```

Python 3.12+ is recommended. Populate the local .env with Meta credentials, the shared AGENTWARD_NODE_TOKEN, and AGENTWARD_INGRESS_URL / AGENTWARD_SANDBOX_URL. Keep secrets out of Git. The deployed private-node setup is described on the next page.

2 / Start the dashboard

```
python -m uvicorn dashboard:app --host 127.0.0.1 \
  --port 8080 --ws-max-size 1024 --ws-max-queue 4
```

Open http://127.0.0.1:8080. Inspect the real provider pre-flight matrix, select an ensemble and choose a benchmark or a source path within the workspace. Retest credentials when configuration changes.

3 / Run a known target

```
python main_pipeline.py benchmark_targets/cwe_78_cmd.js
python main_pipeline.py benchmark_targets/cwe_89_sqli.ts
python main_pipeline.py benchmark_targets/fp_node.mjs
python main_pipeline.py benchmark_targets
```

The final command aggregates the supported Python and Node corpus. Single-file CLI runs replace the latest canonical report; a shared RunReport aggregates a custom subset. Inspect the exact diff and evidence before integration.

> **A useful first comparison**
> **cwe_78_cmd.js** and **cwe_89_sqli.ts** exercise vulnerable command/SQL paths. **fp_node.mjs** is the benign Node control. Passing ingress is expected for all three: the sandbox then distinguishes the tested runtime behavior.

Fixture-mode demonstrations have SIMULATED labels and cannot export verified SARIF fixes. Any permitted Python host execution is test-only; Node still requires Docker. See README.md for the explicit offline-demo commands.

README.md; requirements-dev.txt; docs/INGRESS_GUARD.md

12 / PROJECT GUIDE

Deploy to a supported Linux host

AWS Lightsail is optional. Use a local Linux host or a VM from another provider with the same Docker, filesystem and service-account requirements.

Requirement	Operational contract
Host and account	Rootful Linux Docker without user-namespace remapping; ACL-capable temporary filesystem; service account with Docker access.
Bundled installer	Ubuntu/systemd recipe. USER_NAME is ubuntu; APP_DIR is /home/ubuntu/agentward-node. Adapt these settings for another account or distribution.
Sandbox image	agentward-sandbox:multilang-v3. Recorded: Node 24.22.2 and Python 3.12.16. Dockerfile follows major/minor tags; rebuilds may change patch versions.
Authentication	Random shared AGENTWARD_NODE_TOKEN in root-owned mode-0600 /etc/agentward/node.env and the ignored workstation .env.

Install from the full checkout

Prepare Docker Engine, git, Python venv/pip, curl and ACL support first. On an Ubuntu node as its service account:

```
cd /home/ubuntu
git clone https://github.com/ephraimcrypto21/AgentWard-MuseVerify.git agentward-node
cd agentward-node
sudo docker build -t agentward-sandbox:multilang-v3 \
  -f security/Dockerfile.sandbox .
sudo bash deploy_remote_services.sh
```

The script installs node requirements, provisions the pinned classifier and restarts both services on loopback ports 8000/8001. It preserves an existing token and reuses an existing image tag. For upgrades, update the checkout and explicitly rebuild the image before deploying.

Coordinate versions and verify health

Keep the workstation, services, model contract and image compatible. Check both /health endpoints, then authenticated functional controls. A missing/short node token produces 503; incorrect authorization produces 401. Health alone does not establish live acceptance.

docs/DEPLOYMENT_V3.md; docs/INGRESS_GUARD.md; deploy_remote_services.sh

13 / PROJECT GUIDE

Connect the workstation and node

Choose SSH forwarding or persistent private Tailscale Serve. Services on the same Linux workstation can use loopback ports directly.

Option A / SSH forwarding

```
ssh -N -L 18000:127.0.0.1:8000 \
  -L 18001:127.0.0.1:8001 ubuntu@YOUR_HOST
```

Keep that connection open. Configure these workstation .env values and securely provision the same node token:

```
AGENTWARD_INGRESS_URL=http://127.0.0.1:18000/scan-ingress
AGENTWARD_SANDBOX_URL=http://127.0.0.1:18001/execute
```

Option B / Tailscale Serve within one tailnet

Install Tailscale on both hosts, join the same tailnet and run on the node:

```
tailscale ip -4
sudo tailscale serve --bg --tcp=8000 tcp://127.0.0.1:8000
sudo tailscale serve --bg --tcp=8001 tcp://127.0.0.1:8001
tailscale serve status
```

Use the private address printed above or an enabled MagicDNS name:

```
AGENTWARD_INGRESS_URL=http://<NODE_IP>:8000/scan-ingress
AGENTWARD_SANDBOX_URL=http://<NODE_IP>:8001/execute
```

> **Preserve the private transport boundary**
> Tailscale carries HTTP inside its encrypted network. Restrict tailnet access and retain node bearer authentication; Serve is not public Funnel exposure. For a same-host setup use 127.0.0.1:8000/8001. Restart the dashboard after endpoint changes.

14 / PROJECT GUIDE

Dashboard, API and artifacts

The interface exposes progress and evidence without allowing advisory text to override a verified outcome.

Interface	Operator use
GET /api/health GET /api/languages	Inspect running state, phase identifiers, language extensions and adapter scope.
GET /api/providers POST /api/providers/mode	Inspect pre-flight and select an ensemble. force_refresh=true retests provider configuration.
POST /api/run POST /api/batch	Audit a supported workspace source path or run a bounded benchmark batch.
POST /api/steer POST /api/clean	Add a session restriction; prune temporary workspaces only when idle.
GET /api/metrics GET /api/executive-brief	Read outcomes, usage and timing, plus evidence, assumptions and graphs.
GET /api/sarif WebSocket /ws/telemetry	Export SARIF v2.1.0 or consume replayable events tagged by run.

Review surfaces

The dashboard includes seven phase cards, replayable telemetry, a unified/side-by-side diff viewer, an executive brief and a cross-file call/taint map. Advisory model commentary is labeled and cannot change the outcome established by the gates.

Artifact	Contents
patches/<run-id>/	Accepted candidates and exact diffs; original benchmarks stay unchanged.
reports/benchmark_metrics.json	Per-file results, reviewer votes, pre-flight state, usage and latency.
reports/agentward_sarif_report.sarif	Findings and eligible verified fixes in SARIF format.
reports/executive_brief.md / .html	Evidence-derived assessment with unresolved assumptions and coverage limits.

Reports are written atomically and archived per run. Runtime reports, local workspaces and credentials are excluded from Git. They are operator-writable records, not a signed append-only audit log.

README.md; dashboard.py; reporting.py

15 / PROJECT GUIDE

Codebase and skill integration

The implementation separates orchestration, model access, language rules, service security and evidence contracts.

Module / area	Responsibility
main_pipeline.py / dashboard.py	Orchestration, CLI, session state, API and live telemetry.
ai_provider.py / agentward_stage3.py	Provider discovery, routing, runtime policy and trajectory checks.
language_adapters.py ecmascript_harness.py	Language registry, parsers, contracts, sanitization hooks and deterministic Node probes.
agentward_stage1.py / ingress_server.py ingress_guard.py / ingress_sanitizer.py	Ingress client/service, source-aware inference, sanitization and classifier evidence.
sandbox_server.py / sandbox_protocol.py service_security.py	Constrained execution, protected trace protocol, authentication and bounded schemas.
museverify_stage4.py / security_graph.py semantic_hunter.py / reporting.py	Patch gates, static paths, semantic findings and evidence-derived outputs.
security/ / systemd/ / tests/ / docs/	Pins, Dockerfile, service units, regression corpora, tests and architecture records.

818 pinned community cybersecurity skills

All skills from the requested independent community repository are installed locally for Codex. security/skills.lock.json pins source revision **54a7988** and full-tree hashes. Per-skill licenses and supporting material are preserved. The collection is not affiliated with Anthropic.

Tool-invocation security, container hardening, WebSocket testing, host-header testing and threat-modeling skills informed the controls documented here. This does not claim that every skill was executed or that its procedures establish compliance.

```
python3 tools/manage_cybersecurity_skills.py install
python3 tools/manage_cybersecurity_skills.py verify
```

Guidance has a defined role
The application does not load all 818 skills as runtime instructions or execute their scripts automatically. Installing them does not add language adapters or authorize testing outside the operator's requested scope.

docs/SKILLS.md; security/skills.lock.json; repository modules at 2d52f29

16 / PROJECT GUIDE

What has been verified

Application snapshot 2d52f29; evidence reviewed 1 November 2026. Each result has a specific date, mode and coverage.

Evidence set	Recorded result	What the result supports
Test suite 1 November	**99 passed** Zero skips	Includes real Docker and corpus suites. Unit model fixtures do not constitute live-provider evidence.
Live Node corpus 1 November	**13 targets** 8 / 4 / 1 outcomes	Real classifier, Meta Spark/Wonder and authenticated remote Docker: eight repairs, four false positives, one inconclusive weakness.
Adapter and baseline checks	**6 TS transforms** **17 exports**	All 13 targets parse and pass ingress unchanged. Six TypeScript sources erase/reparse/scan; every export returns normally in Docker.
Classifier regression 31 October	**78 / 78** 45 benign + 33 malicious	Real pinned-model inference. Benign cases unchanged; malicious cases blocked or sanitized and rescanned. Development regression sets.
Remote deployment 31 October	**25 groups**	Auth, language/hash binding, all seven extensions and isolation. Repair controls in this historical record used fixtures.
Skill integrity	**818 pinned trees**	Installed collection matches the source revision and full-tree hashes; this is not an audit of every procedure.

How to read the classifier result

The 78 cases include the reported false positives, normal object literals and exports, multilingual instructions, base64, long inputs, fake metadata and repeated documentation. These corpora were used during policy development. The result is regression evidence, not an accuracy estimate from an untouched holdout population.

> **Current live evidence and historical checks**
> The latest corpus repairs were generated by live Meta Spark and independently reviewed by live Meta Wonder. Optional providers are covered by router tests, not claimed as participants in this live run. Historical fixture and browser records retain their original scope.

All reported counts come from the repository acceptance records and retained local evidence. They do not estimate industry-wide alert backlogs, model failure rates or general detection accuracy.

docs/CORPUS_ACCEPTANCE_2026-11-01.md; docs/INGRESS_GUARD.md; docs/DEPLOYMENT_V3.md

17 / PROJECT GUIDE

Live Node corpus: every outcome

All six Node extensions were checked. Detailed run IDs, source hashes, initial failures and accepted patch paths are in the dated corpus acceptance record.

Target under benchmark_targets/	Final live outcome
cwe_78_cmd.js	REMEDIATED
cwe_78_cmd_mjs.mjs	REMEDIATED
cwe_78_cmd_alias.js	REMEDIATED
cwe_78_cmd_ping.cts	REMEDIATED
cwe_89_sqli.ts	REMEDIATED
cwe_89_sqli_cjs.cts	REMEDIATED
cwe_89_sqli_like.cjs	REMEDIATED
cwe_89_sqli_login.mts	REMEDIATED
fp_node.mjs	FALSE_POSITIVE
fp_node_cjs.cjs	FALSE_POSITIVE
fp_typescript_safe.ts	FALSE_POSITIVE
fp_typescript_safe.mts	FALSE_POSITIVE
cwe_338_weak_token.cjs	INCONCLUSIVE

Eight repairs neutralized 24 self-play scenarios with zero survivors. Saved patches preserve extensions and signatures, match protected regression hashes, and appear as eight verified SARIF fixes. All four benign controls made zero patch-synthesis or patch-verification calls.

The diagnostic CTS target now uses printf instead of network-dependent ping, preserving its command-injection defect. Two initial classifier-capacity failures remain archived; both targets passed reruns after bounded HTTP 429 retry handling.

Retain the untested weakness
SQL probes do not verify Math.random token unpredictability. The weak-token case retains CWE-338, has no verified fix and remains INCONCLUSIVE. The combined SARIF invocation therefore reports executionSuccessful=false.

docs/CORPUS_ACCEPTANCE_2026-11-01.md; reports/corpus-2026-11-01/ (local evidence)

18 / PROJECT GUIDE

Interpreting results and limits

Use each outcome as evidence for the behavior and coverage actually checked. A completed run is not a formal proof of program safety.

Result or signal	Interpretation
Ingress ALLOW / SANITIZED	The source may continue to the remaining gates. This is not a vulnerability verdict.
Ingress BLOCKED / UNVERIFIED	A classified attack was denied, or required evidence was absent/stale. The pipeline records an ingress-blocked outcome when live ingress cannot pass.
REMEDIATED	Applicable gates and tested repair behavior passed with production evidence. Review the diff and stated assumptions before integrating it.
FALSE_POSITIVE	Safe-path evidence and Phase 3 policy passed; patch synthesis was skipped. Node also requires no static finding outside its probe CWEs. This differs from an ingress classifier false positive.
SIMULATED_*	Fixture-based demonstration. It cannot establish a production repair or export a verified SARIF fix.
INCONCLUSIVE / rejected / failed	Evidence, policy, repair or regression requirements were not satisfied. Missing evidence is never an implicit pass.

Coverage that remains bounded

Detection: unfamiliar instructions can evade the learned model; benign code can still be misclassified. New regressions should expand the corpus and trigger review of the pinned policy and model.

Program analysis: Python propagation is bounded; Node taint analysis is intraprocedural. Reflection, closures, dynamic dispatch, external libraries and Node cross-file flows need additional review. CodeShield coverage is narrower than a complete security audit.

Runtime and operations: selected probes and syscall traces cover observed behavior. The kernel, daemon, service, tracer and operator remain trusted. The shared token identifies an authorized workstation, not an individual user; provider data handling remains an operator decision.

> **A useful next validation step**
> For a new production target, add representative functional probes and run the full live pipeline with fresh provider pre-flight. Expand adapters explicitly for new languages, application shapes and vulnerability families. These are extension directions, not shipped capabilities.

docs/THREAT_MODEL.md; docs/LANGUAGES.md; docs/INGRESS_GUARD.md

19 / PROJECT GUIDE

References and reproducibility

The PDF is a reviewed snapshot. Versioned repository documents and their implementation provide the detailed source of each claim.

Reference	Use it for
README.md	Setup, provider configuration, API routes and complete verification commands.
docs/INGRESS_GUARD.md	Classifier policy, model pin, regression evaluation and deployment.
docs/THREAT_MODEL.md	Trust boundaries, threat inventory and residual limits.
docs/LANGUAGES.md	Adapter contracts, syntax and execution limits.
docs/DEPLOYMENT_V3.md	Recorded remote acceptance and coordinated rollback.
1 November corpus acceptance	All 13 live results, 99-test record, retries and artifact verification.
docs/SKILLS.md	Community provenance, exact pin and applied skills.

Detailed live acceptance JSON remains in ignored local reports. The published documents summarize the evidence without embedding credentials, private endpoints or provider transcripts.

Reproduce checks and rebuild this guide

```
AGENTWARD_TEST_DOCKER=1 \
  SANDBOX_IMAGE=agentward-sandbox:multilang-v3 \
  python -m unittest discover -v
python -m pip check
python -m pip install -r requirements-docs.txt
python tools/build_project_guide.py \
  --output AgentWard_MuseVerify_Project_Guide.pdf
```

The generator writes the PDF and same-stem HTML companion from one reviewed source. Without --output it updates output/pdf/. No browser, live provider or node call is needed to build the guide.

README.md lists real Docker and live E2E checks. docs/INGRESS_GUARD.md lists all three classifier regression corpora; those commands need the provisioned node dependencies and pinned model. Rebuilding this guide does not revalidate a deployment.

Terms: AST: abstract syntax tree. BYOK: bring your own API key. CWE: Common Weakness Enumeration. SARIF: Static Analysis Results Interchange Format. Positive control: an original-code test that must reproduce the expected vulnerable behavior.

Application revision: 2d52f29a585abf5c4dc241bdda118b5834c63126